RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105

AI23331 Fundamentals of Machine Learning
Laboratory Record Note Book

Name :	Kamalesh S P
Year / Branch / Section :	III – CSE – C
University Register No. :	2116230701138
College Roll No. :	230701138
Semester :	VI
Academic Year :	2025 – 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AI23331 Fundamentals of Machine Learning

NAME	Kamalesh S P
ROLL NO	230701138
YEAR	III
DEPT	CSE

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……..……KAMALESH S P…….

Academic Year: …2025 – 2026… Semester: …VI… Branch: ….CSE – C….

Register No.	**2116230701138**

Certified that this is the bonafide record of work done by the above student in the* FUNDAMENTALS OF MACHINE LEARNING *Laboratory during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on 19/05/2026.

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1a	A python program to implement Data Cleaning.	**06.01.2026**	
1b	A python program to implement univariate regression, bivariate regression and multivariate regression.	**28.01.2026**	
2	A python program to implement Simple linear regression using Least Square Method.	**10.02.2026**	
3	A python program to implement logistic model.	**10.03.2026**	
4	A python program to implement single layer perceptron.	**17.03.2026**	 230701138 - SCAN ME
5	A python program to implement multi-layer perceptron with back propagation.	**24.03.2026**	
6	A python program to do face recognition using SVM classifier.	**30.03.2026**	
7	A python program to implement decision tree.	**31.03.2026**	
8	A python program to implement boosting.	**07.04.2026**	
9a	A python program to implement KNN.	**07.04.2026**	
9b	A python program to implement K-means.	**15.04.2026**	
10	A python program to implement dimensionality reduction – PCA.	**15.04.2026**	

Ex No: 1a Date: 06/01/2026	**Dataset Exploration (EDA) & Regression Analysis**

Aim:

To perform exploratory data analysis on the Iris dataset and apply univariate, bivariate, and multivariate regression analysis.

Algorithm:

1. Import required libraries (pandas, numpy, matplotlib, seaborn, sklearn).
2. Load the Iris dataset using sklearn or read from CSV.
3. Perform Univariate Analysis: scatter plots per species for single features.
4. Perform Bivariate Analysis: FacetGrid scatter plot of two features.
5. Perform Multivariate Analysis: Pair plot of all features coloured by species.
6. Apply Univariate Linear Regression (petal length → sepal length), calculate R^2.
7. Apply Bivariate Linear Regression (petal length + petal width → sepal length), calculate R^2.
8. Apply Multivariate Linear Regression (all features → sepal length), calculate R^2.
9. Plot regression results and display coefficients, intercepts, and R^2 scores.

Code:

```
# Step 1: Import necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.datasets import load_iris
# Step 2: Read the dataset
# Load Iris dataset from sklearn
iris = load_iris()

# Convert to DataFrame
data = pd.DataFrame(iris.data, columns=iris.feature_names)
data['target'] = iris.target
# Step 3: Prepare the data
# Using sepal length as dependent variable
y = data['sepal length (cm)']
# ------------------ UNIVARIATE REGRESSION ------------------
# Step 4: Univariate Regression
X_uni = data[['petal length (cm)']]

model_uni = LinearRegression()
model_uni.fit(X_uni, y)

y_pred_uni = model_uni.predict(X_uni)
r2_uni = r2_score(y, y_pred_uni)
# ------------------ BIVARIATE REGRESSION ------------------
# Step 5: Bivariate Regression
X_bi = data[['petal length (cm)', 'petal width (cm)']]

model_bi = LinearRegression()
model_bi.fit(X_bi, y)

y_pred_bi = model_bi.predict(X_bi)
r2_bi = r2_score(y, y_pred_bi)
# ------------------ MULTIVARIATE REGRESSION ------------------
# Step 6: Multivariate Regression
X_multi = data.drop(columns=['target'])

model_multi = LinearRegression()
model_multi.fit(X_multi, y)

y_pred_multi = model_multi.predict(X_multi)
```

```
r2_multi = r2_score(y, y_pred_multi)
# ------------------ DISPLAY Plot ------------------
# Step 7: Plot the results:

# Plot Univariate Regression
plt.figure()
plt.scatter(X_uni, y, color='blue', label='Actual Data')
plt.plot(X_uni, y_pred_uni, color='red', label='Regression Line')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Sepal Length (cm)')
plt.title('Univariate Regression')
plt.legend()
plt.show()

# Plot Bivariate Regression (3D)
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X_bi.iloc[:, 0], X_bi.iloc[:, 1], y, color='blue')

ax.set_xlabel('Petal Length (cm)')
ax.set_ylabel('Petal Width (cm)')
ax.set_zlabel('Sepal Length (cm)')
ax.set_title('Bivariate Regression')
ax.set_box_aspect(None, zoom=0.85)
plt.show()

# Plot Multivariate Regression (Predicted vs Actual)
plt.figure()
plt.scatter(y, y_pred_multi, color='green')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Multivariate Regression')
plt.show()
# ------------------ DISPLAY RESULTS ------------------
# Step 8: Display Results
print("UNIVARIATE REGRESSION")
print("Coefficient:", model_uni.coef_)
print("Intercept:", model_uni.intercept_)
print("R-squared:", r2_uni)

print("\nBIVARIATE REGRESSION")
print("Coefficients:", model_bi.coef_)
print("Intercept:", model_bi.intercept_)
print("R-squared:", r2_bi)

print("\nMULTIVARIATE REGRESSION")
print("Coefficients:", model_multi.coef_)
print("Intercept:", model_multi.intercept_)
print("R-squared:", r2_multi)
```

Output:

```
<Figure size 640x480 with 1 Axes>
<Figure size 1000x800 with 1 Axes>
<Figure size 640x480 with 1 Axes>
UNIVARIATE REGRESSION
Coefficient: [0.40892228]
Intercept: 4.306603415047579
R-squared: 0.759954645772515

BIVARIATE REGRESSION
Coefficients: [ 0.54177715 -0.31955056]
Intercept: 4.190582428651588
R-squared: 0.7662612975425306

MULTIVARIATE REGRESSION
Coefficients: [ 1.00000000e+00  5.91759465e-17 -4.42355329e-17 -1.54405799e-16]
```

```
Intercept: -2.6645352591003757e-15
R-squared: 1.0
```

Result:
The Iris dataset was successfully explored through univariate, bivariate and multivariate analyses. Regression models were fitted with R^2 of 0.76 (univariate), 0.77 (bivariate), and 1.0 (multivariate), demonstrating that combining all features perfectly explains sepal length variance.

Ex No: 1b Date: 28/01/2026	Data Cleaning & Exploratory Data Analysis (EDA)

Aim:

To perform data cleaning, outlier detection using IQR, correlation analysis, feature scaling using normalization and standardization on the Diabetes dataset.

Algorithm:

1. Import libraries: pandas, numpy, sklearn, matplotlib, seaborn.
2. Load the Diabetes dataset from CSV.
3. Inspect data structure using df.info() and check for missing values.
4. Visualize outliers using box plots for each feature.
5. Remove outliers using the IQR method on the Insulin column.
6. Compute correlation matrix and visualize using a heatmap.
7. Visualize the target variable distribution using a pie chart.
8. Separate features (X) and target (y).
9. Apply Min-Max Normalization using MinMaxScaler.
10. Apply Standard Scaling using StandardScaler.

Code:

```
# Step 1: Import Libraries and Load Dataset
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv('diabetes.csv')
df.head()
# Step 2: Inspect Data Structure and Check Missing Values
df.info()
df.isnull().sum()
df.describe()

fig, axs = plt.subplots(len(df.columns), 1, figsize=(7, 18), dpi=95)
for i, col in enumerate(df.columns):
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)
plt.tight_layout()
plt.show()
# Step 4: Remove Outliers Using the Interquartile Range (IQR) Method
q1, q3 = np.percentile(df['Insulin'], [25, 75])
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
clean_df = df[df['Insulin'].between(lower_bound, upper_bound)]
# Step 5: Correlation Analysis
corr = df.corr()
plt.figure(dpi=130)
sns.heatmap(corr, annot=True, fmt="0.2f", cmap='coolwarm')
plt.show()

corr['Outcome'].sort_values(ascending=False)
# Step 6: Visualize Target Variable Distribution
plt.pie(df['Outcome'].value_counts(), labels=['No Diabetes', 'Diabetes'],
autopct='%0.f%%', shadow=True)
plt.title('Diabetes Outcome Distribution')
plt.show()
# Step 7: Separate Features and Target Variable
X = df.drop(columns=['Outcome'])
y = df['Outcome']
```

```
# Step 8: Feature Scaling: Normalization and Standardization
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)
X_normalized[:5]
scaler = StandardScaler()
X_standardized = scaler.fit_transform(X)
X_standardized[:5]
```

Output:

```
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
<Figure size 665x1710 with 9 Axes>
<Figure size 832x624 with 2 Axes>
Outcome                     1.000000
Glucose                     0.466581
BMI                         0.292695
Age                         0.238356
Pregnancies                 0.221898
DiabetesPedigreeFunction    0.173844
Insulin                     0.130548
SkinThickness               0.074752
BloodPressure               0.065068
Name: Outcome, dtype: float64
<Figure size 640x480 with 1 Axes>
array([[0.35294118, 0.74371859, 0.59016393, 0.35353535, 0.        ,
        0.50074516, 0.23441503, 0.48333333],
       [0.05882353, 0.42713568, 0.54098361, 0.29292929, 0.        ,
        0.39642325, 0.11656704, 0.16666667],
       [0.47058824, 0.91959799, 0.52459016, 0.        , 0.        ,
        0.34724292, 0.25362938, 0.18333333],
```

```
[0.05882353, 0.44723618, 0.54098361, 0.23232323, 0.11111111,
 0.41877794, 0.03800171, 0.        ],
[0.        , 0.68844221, 0.32786885, 0.35353535, 0.19858156,
 0.64232489, 0.94363792, 0.2       ]])
```

Result:

Data cleaning was performed successfully. Outliers were detected and removed using IQR. Glucose showed the highest correlation (0.47) with the Outcome. Both normalization and standardization were applied successfully to scale features.

Ex No: 2 Date: 10/02/2026	Linear Regression

Aim:

To implement Linear Regression using both the manual mathematical approach and sklearn library to predict salary from years of experience.

Algorithm:

1. Import numpy, pandas, and sklearn LinearRegression.
2. Load Experience-Salary dataset.
3. Manual Method: Compute slope (m) and intercept (c) using least-squares formulas.
4. Predict salary using manual model: y_pred = m*x + c.
5. Convert continuous predictions to binary class using mean threshold.
6. Compute manual confusion matrix, accuracy, precision, recall, F1-score.
7. sklearn Method: Fit LinearRegression model on training data.
8. Predict salary using sklearn model.
9. Compute sklearn metrics using accuracy_score, confusion_matrix, classification_report.
10. Compare results of both models.

Code:

```
# IMPORT LIBRARIES
import numpy as np
import pandas as pd

from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# LOAD DATA
data = pd.read_csv('Experience-Salary.csv')

x = data["exp(in months)"].values
y = data["salary(in thousands)"].values

n = len(x)
print("Total samples:", n)
# MODEL 1: MANUAL LINEAR REGRESSION (FORMULA METHOD)
# SECTION 1: Manual Training
sum_x = np.sum(x)
sum_y = np.sum(y)
sum_x2 = np.sum(x ** 2)
sum_xy = np.sum(x * y)

m_manual = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)
c_manual = (sum_y - m_manual * sum_x) / n
# SECTION 2: Manual Model Output (VISIBLE)
print("===== MANUAL LINEAR REGRESSION MODEL =====")
print(f"Slope (m)     : {m_manual}")
print(f"Intercept (c) : {c_manual}")
print(f"Equation      : Salary = {m_manual:.2f} * Experience + {c_manual:.2f}")
# SECTION 3: Manual Predictions
y_pred_manual = m_manual * x + c_manual
# SECTION 4: Manual → Classification

# Single, shared rule
# Salary ≥ mean → Class 1
# Salary < mean → Class 0
threshold = np.mean(y)

y_actual_class = np.where(y >= threshold, 1, 0)
y_pred_class_manual = np.where(y_pred_manual >= threshold, 1, 0)
# SECTION 5: MANUAL MODEL METRICS (FORMULA ONLY)

TP = np.sum((y_actual_class == 1) & (y_pred_class_manual == 1))
```

```
TN = np.sum((y_actual_class == 0) & (y_pred_class_manual == 0))
FP = np.sum((y_actual_class == 0) & (y_pred_class_manual == 1))
FN = np.sum((y_actual_class == 1) & (y_pred_class_manual == 0))

cm_manual = np.array([[TN, FP], [FN, TP]])

accuracy_manual = (TP + TN) / (TP + TN + FP + FN)

precision_0 = TN / (TN + FN) if (TN + FN) != 0 else 0
recall_0    = TN / (TN + FP) if (TN + FP) != 0 else 0
f1_0        = (2 * precision_0 * recall_0) / (precision_0 + recall_0) if
(precision_0 + recall_0) != 0 else 0

precision_1 = TP / (TP + FP) if (TP + FP) != 0 else 0
recall_1    = TP / (TP + FN) if (TP + FN) != 0 else 0
f1_1        = (2 * precision_1 * recall_1) / (precision_1 + recall_1) if
(precision_1 + recall_1) != 0 else 0

# Supports
support_0 = np.sum(y_actual_class == 0)
support_1 = np.sum(y_actual_class == 1)
total = support_0 + support_1

# Macro averages
macro_precision = (precision_0 + precision_1) / 2
macro_recall = (recall_0 + recall_1) / 2
macro_f1 = (f1_0 + f1_1) / 2

# Weighted averages
weighted_precision = (
    (precision_0 * support_0 + precision_1 * support_1) / total
)
weighted_recall = (
    (recall_0 * support_0 + recall_1 * support_1) / total
)
weighted_f1 = (
    (f1_0 * support_0 + f1_1 * support_1) / total
)
# SECTION 5: Manual Model Metrics (sklearn)
print("--- MANUAL MODEL METRICS ---")

print("Accuracy:", round(accuracy_manual, 2))

print("\nConfusion Matrix:\n", cm_manual)

print("\nClassification Report (Manual):\n")
print(f"{'Class':<10}{'Precision':<12}{'Recall':<10}{'F1-
score':<10}{'Support':<10}")
print("-" * 55)
print(f"{'0':<10}{precision_0:<12.2f}{recall_0:<10.2f}{f1_0:<10.2f}{support_0:<10}"
)
print(f"{'1':<10}{precision_1:<12.2f}{recall_1:<10.2f}{f1_1:<10.2f}{support_1:<10}"
)
print("-" * 55)
print(f"{'Accuracy':<34}{accuracy_manual:<10.2f}{total:<10}")
print(f"{'Macro
Avg':<10}{macro_precision:<12.2f}{macro_recall:<10.2f}{macro_f1:<10.2f}{total:<10}"
)
print(f"{'Weighted
Avg':<10}{weighted_precision:<12.2f}{weighted_recall:<10.2f}{weighted_f1:<10.2f}{to
tal:<10}")
# MODEL 2: SKLEARN LINEAR REGRESSION (BUILT-IN)
# SECTION 6: sklearn Training
X = x.reshape(-1, 1)

lr_model = LinearRegression()
lr_model.fit(X, y)
```

```
m_sklearn = lr_model.coef_[0]
c_sklearn = lr_model.intercept_
# SECTION 7: sklearn Model Output (VISIBLE)
print("===== SKLEARN LINEAR REGRESSION MODEL =====")
print(f"Slope (m)     : {m_sklearn}")
print(f"Intercept (c) : {c_sklearn}")
print(f"Equation      : Salary = {m_sklearn:.2f} * Experience + {c_sklearn:.2f}")
# SECTION 8: sklearn Predictions
y_pred_sklearn = lr_model.predict(X)
# SECTION 9: sklearn → Classification
y_pred_class_sklearn = np.where(y_pred_sklearn >= threshold, 1, 0)
# SECTION 10: sklearn Model Metrics (sklearn)
print("===== SKLEARN MODEL METRICS =====")

accuracy_sklearn = accuracy_score(y_actual_class, y_pred_class_sklearn)
cm_sklearn = confusion_matrix(y_actual_class, y_pred_class_sklearn)
report_sklearn = classification_report(y_actual_class, y_pred_class_sklearn)

print("Accuracy:", round(accuracy_sklearn, 2))
print("\nConfusion Matrix:\n", cm_sklearn)
print("\nClassification Report:\n", report_sklearn)
# Run Time Input
print("===== RUNTIME PREDICTION USING BOTH MODELS =====")

exp_input = float(input("Enter years of experience: "))

salary_manual = m_manual * exp_input + c_manual

salary_sklearn = lr_model.predict([[exp_input]])[0]

print(f"\nPrediction Results for {exp_input} Year(s) of Expirence:")
print("-----------------------------------")
print(f"Manual Linear Regression Prediction  : {salary_manual:.2f} Thousand")
print(f"sklearn Linear Regression Prediction : {salary_sklearn:.2f} Thousand")
print("-----------------------------------")
```

Output:

```
Total samples: 1000
===== MANUAL LINEAR REGRESSION MODEL =====
Slope (m)     : 0.8228573477761276
Intercept (c) : 5.198560811223339
Equation      : Salary = 0.82 * Experience + 5.20
--- MANUAL MODEL METRICS ---
Accuracy: 0.82

Confusion Matrix:
 [[414  85]
 [ 98 403]]

Classification Report (Manual):

Class     Precision   Recall    F1-score  Support
-------------------------------------------------------
0         0.81        0.83      0.82      499
1         0.83        0.80      0.81      501
-------------------------------------------------------
Accuracy                          0.82      1000
Macro Avg 0.82        0.82      0.82      1000
Weighted Avg0.82        0.82      0.82      1000
===== SKLEARN LINEAR REGRESSION MODEL =====
Slope (m)     : 0.8228573477761271
Intercept (c) : 5.198560811223356
Equation      : Salary = 0.82 * Experience + 5.20
===== SKLEARN MODEL METRICS =====
Accuracy: 0.82

Confusion Matrix:
```

```
 [[414  85]
 [ 98 403]]

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.83      0.82       499
           1       0.83      0.80      0.81       501

    accuracy                           0.82      1000
   macro avg       0.82      0.82      0.82      1000
weighted avg       0.82      0.82      0.82      1000
===== RUNTIME PREDICTION USING BOTH MODELS =====

Prediction Results for 10.0 Year(s) of Expirence:
-----------------------------------
Manual Linear Regression Prediction  : 13.43 Thousand
sklearn Linear Regression Prediction : 13.43 Thousand
-----------------------------------
```

Result:

Linear Regression was implemented using both manual formula and sklearn. Both approaches yielded identical results: Slope ≈ 0.82, Intercept ≈ 5.20. Accuracy of 82% was achieved using a mean-threshold classification.

Ex No: 3 Date: 10/03/2026	Logistic Regression

Aim:

To implement Logistic Regression using manual gradient descent and sklearn to classify student pass/fail based on study hours and previous exam scores.

Algorithm:

1. Import pandas, numpy, sklearn modules.
2. Load student_exam_data.csv.
3. Select features: Study Hours, Previous Exam Score; target: Pass/Fail.
4. Split data into train and test sets (80:20).
5. Scale features using StandardScaler.
6. Manual Method: Define sigmoid function.
7. Initialize weights and bias; run gradient descent for 1000 epochs.
8. Predict using manual model; compute confusion matrix and metrics.
9. sklearn Method: Train LogisticRegression; predict on test set.
10. Compare manual and sklearn metrics.

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
# 1. Load Dataset
data = pd.read_csv('student_exam_data.csv')
# 2. Select Features and Target
X = data[["Study Hours", "Previous Exam Score"]].values
y = data["Pass/Fail"].values
# 3. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
# 4. Feature Scaling (MANDATORY for Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# MANUAL LOGISTIC REGRESSION
# Sigmoid Function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
# Initialize Parameters
weights = np.zeros(X_train_scaled.shape[1])
bias = 0
learning_rate = 0.01
epochs = 1000
# Gradient Descent (Manual Training)
for _ in range(epochs):
    linear_model = np.dot(X_train_scaled, weights) + bias
    y_pred = sigmoid(linear_model)

    dw = (1 / len(y_train)) * np.dot(X_train_scaled.T, (y_pred - y_train))
    db = (1 / len(y_train)) * np.sum(y_pred - y_train)

    weights -= learning_rate * dw
    bias -= learning_rate * db
# Manual Prediction & Thresholding
y_test_pred_prob_manual = sigmoid(np.dot(X_test_scaled, weights) + bias)
y_test_pred_manual = (y_test_pred_prob_manual >= 0.5).astype(int)
# Manual Confusion Matrix
```

```
TP = FP = TN = FN = 0

for actual, predicted in zip(y_test, y_test_pred_manual):
    if actual == 1 and predicted == 1:
        TP += 1
    elif actual == 0 and predicted == 0:
        TN += 1
    elif actual == 0 and predicted == 1:
        FP += 1
    elif actual == 1 and predicted == 0:
        FN += 1

confusion_matrix_manual = np.array([[TN, FP], [FN, TP]])

# Manual Accuracy Calculation
accuracy_manual = (TP + TN) / (TP + TN + FP + FN)

# Manual Precision, Recall, F1-score
precision_0 = TN / (TN + FN) if (TN + FN) != 0 else 0
recall_0 = TN / (TN + FP) if (TN + FP) != 0 else 0
f1_0 = (2 * precision_0 * recall_0) / (precision_0 + recall_0) if (precision_0 +
recall_0) != 0 else 0

precision_1 = TP / (TP + FP) if (TP + FP) != 0 else 0
recall_1 = TP / (TP + FN) if (TP + FN) != 0 else 0
f1_1 = (2 * precision_1 * recall_1) / (precision_1 + recall_1) if (precision_1 +
recall_1) != 0 else 0

# Manual Classification Report
support_0 = TN + FP
support_1 = TP + FN
total = support_0 + support_1

macro_precision = (precision_0 + precision_1) / 2
macro_recall = (recall_0 + recall_1) / 2
macro_f1 = (f1_0 + f1_1) / 2

weighted_precision = ((precision_0 * support_0) + (precision_1 * support_1)) /
total
weighted_recall = ((recall_0 * support_0) + (recall_1 * support_1)) / total
weighted_f1 = ((f1_0 * support_0) + (f1_1 * support_1)) / total
# Manual Model Metrics
print("===== MANUAL LOGISTIC REGRESSION =====")
print("Accuracy (Manual):", round(accuracy_manual, 2))

print("\nConfusion Matrix (Manual):\n", confusion_matrix_manual)

print("\nClassification Report (Manual):\n")
print(f"{'Class':<10}{'Precision':<12}{'Recall':<10}{'F1-
score':<10}{'Support':<10}")
print("-" * 55)
print(f"{'0':<10}{precision_0:<12.2f}{recall_0:<10.2f}{f1_0:<10.2f}{support_0:<10}"
)
print(f"{'1':<10}{precision_1:<12.2f}{recall_1:<10.2f}{f1_1:<10.2f}{support_1:<10}"
)
print("-" * 55)
print(f"{'Accuracy':<34}{accuracy_manual:<10.2f}{total:<10}")
print(f"{'Macro
Avg':<10}{macro_precision:<12.2f}{macro_recall:<10.2f}{macro_f1:<10.2f}{total:<10}"
)
print(f"{'Weighted
Avg':<10}{weighted_precision:<12.2f}{weighted_recall:<10.2f}{weighted_f1:<10.2f}{to
tal:<10}")
# SKLEARN LOGISTIC REGRESSION
# Train Logistic Regression Model
log_model = LogisticRegression()
log_model.fit(X_train_scaled, y_train)
# 6. sklearn Prediction & Threshold
```

```
y_test_pred_prob_sklearn = log_model.predict_proba(X_test_scaled)[:, 1]
y_test_pred_sklearn = (y_test_pred_prob_sklearn >= 0.5).astype(int)
# 7. sklearn Model Metrics
print("===== SKLEARN LOGISTIC REGRESSION =====")
print("Accuracy:", accuracy_score(y_test, y_test_pred_sklearn))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_test_pred_sklearn))

print("\nClassification Report:\n", classification_report(y_test,
y_test_pred_sklearn))
# Runtime Test Input
study_hours_input = float(input("Enter No. of study hours: "))
previous_marks_input = float(input("Enter previous exam marks: "))

runtime_input = np.array([[study_hours_input, previous_marks_input]])
runtime_input_scaled = scaler.transform(runtime_input)
# Runtime Prediction Output
# Manual Model
manual_prob = sigmoid(np.dot(runtime_input_scaled, weights) + bias)[0]
manual_result = "Pass" if manual_prob >= 0.5 else "Fail"

# sklearn Model
sklearn_prob = log_model.predict_proba(runtime_input_scaled)[0][1]
sklearn_result = "Pass" if sklearn_prob >= 0.5 else "Fail"

print("===== RUNTIME PREDICTION USING BOTH MODELS =====\n")
print(f"Prediction Results for {study_hours_input} Hour(s) of Study:")
print("-----------------------------------")
print(f"Manual Logistic Regression Prediction  : {manual_result}
({manual_prob:.2f})")
print(f"sklearn Logistic Regression Prediction : {sklearn_result}
({sklearn_prob:.2f})")
print("-----------------------------------")
```

Output:

```
===== MANUAL LOGISTIC REGRESSION =====
Accuracy (Manual): 0.83

Confusion Matrix (Manual):
 [[117  17]
 [ 17  49]]

Classification Report (Manual):

Class     Precision   Recall    F1-score  Support
-------------------------------------------------------
0         0.87        0.87      0.87      134
1         0.74        0.74      0.74      66
-------------------------------------------------------
Accuracy                          0.83      200
Macro Avg 0.81        0.81      0.81      200
Weighted Avg0.83        0.83      0.83      200
LogisticRegression()
===== SKLEARN LOGISTIC REGRESSION =====
Accuracy: 0.83

Confusion Matrix:
 [[117  17]
 [ 17  49]]

Classification Report:
               precision    recall  f1-score   support

           0       0.87      0.87      0.87       134
           1       0.74      0.74      0.74        66

    accuracy                           0.83       200
```

```
   macro avg       0.81      0.81      0.81       200
weighted avg       0.83      0.83      0.83       200
===== RUNTIME PREDICTION USING BOTH MODELS =====

Prediction Results for 9.0 Hour(s) of Study:
-----------------------------------
Manual Logistic Regression Prediction  : Fail (0.09)
sklearn Logistic Regression Prediction : Fail (0.00)
-----------------------------------
```

Result:

Logistic Regression was implemented using manual gradient descent and sklearn. Both achieved 83% accuracy on the student exam dataset with identical confusion matrices, confirming correctness of the manual implementation.

Ex No: 4 Date: 17/03/2026	Single Layer Perceptron

Aim:

To implement a Single Layer Perceptron from scratch and train it to simulate AND, OR, NAND, and NOR logical gates.

Algorithm:

1. Import pandas and numpy.
2. Load logical_gate.csv dataset.
3. Clean data: remove invalid rows, keep binary inputs only.
4. Map text labels (zero/one) to numeric values.
5. Define step activation function.
6. Define perceptron training function using weight update rule.
7. Define testing function to generate predictions.
8. Train perceptron for each gate: AND, OR, NAND, NOR.
9. Display learned weights, bias, and predictions for each input combination.

Code:

```
import pandas as pd
import numpy as np
# ------------------------------
# 1 Load Dataset
# ------------------------------
data = pd.read_csv("logical_gate.csv")
# ------------------------------
# 2 Data Cleaning
# ------------------------------
# remove invalid rows
data = data[data["AND_Gate"] != "invalid"]

# keep only binary inputs
data = data[(data["Value_A"] <= 1) & (data["Value_B"] <= 1)]

# convert text to numbers
replace_map = {"zero":0, "one":1}

data["AND_Gate"] = data["AND_Gate"].map(replace_map)
data["OR_Gate"] = data["OR_Gate"].map(replace_map)
data["NAND_Gate"] = data["NAND_Gate"].map(replace_map)
data["NOR_Gate"] = data["NOR_Gate"].map(replace_map)

# remove rows with missing values
data = data.dropna()
# ------------------------------
# 3 Input Features
# ------------------------------
X = data[["Value_A","Value_B"]].values
# ------------------------------
# 4 Step Activation Function
# ------------------------------
def step(z):
    return 1 if z >= 0 else 0
# ------------------------------
# 5 Perceptron Training Function
# ------------------------------
def train_perceptron(X, y, lr=0.1, epochs=10):

    weights = np.zeros(X.shape[1])
    bias = 0

    for _ in range(epochs):
```

```
        for i in range(len(X)):

            z = np.dot(X[i], weights) + bias
            y_pred = step(z)

            error = y[i] - y_pred

            weights = weights + lr * error * X[i]
            bias = bias + lr * error

    return weights, bias
# ----------------------------
# 6 Testing Function
# ----------------------------
def test_model(X, weights, bias):

    predictions = []

    for x in X:
        z = np.dot(x, weights) + bias
        predictions.append(step(z))

    return predictions
# ----------------------------
# 7 Train Models for All Gates
# ----------------------------
gates = ["AND_Gate","OR_Gate","NAND_Gate","NOR_Gate"]

for gate in gates:

    print("\n==========", gate, "==========")

    y = data[gate].values

    weights, bias = train_perceptron(X, y)

    predictions = test_model(X, weights, bias)

    print("Weights:", weights)
    print("Bias:", bias)

    print("\nPredictions")
    unique_X = np.unique(X, axis=0)
    for x in unique_X:
        z = np.dot(x, weights) + bias
        print(x, "->", step(z))
```

Output:

```
========== AND_Gate ==========
Weights: [0.1 0.2]
Bias: -0.20000000000000004

Predictions
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

========== OR_Gate ==========
Weights: [0.1 0.1]
Bias: -0.1

Predictions
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1
```

```
========== NAND_Gate ==========
Weights: [-0.1 -0.2]
Bias: 0.2

Predictions
[0 0] -> 1
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0

========== NOR_Gate ==========
Weights: [-0.1 -0.1]
Bias: 0.0

Predictions
[0 0] -> 1
[0 1] -> 0
[1 0] -> 0
[1 1] -> 0
```

Result:

Single Layer Perceptron was successfully trained to simulate AND, OR, NAND, and NOR gates. All gates produced correct outputs, demonstrating the perceptron's ability to learn linearly separable functions.

Ex No: 5 Date: 24/03/2026	Multi Layer Perceptron (Backpropagation)

Aim:

To implement a Multi-Layer Perceptron using backpropagation to solve the XOR problem which is not linearly separable.

Algorithm:

1. Import pandas and numpy.
2. Load XOR_Dataset.csv with features X, Y and target Z.
3. Define sigmoid and sigmoid_derivative functions.
4. Initialize weights W1, W2 and biases b1, b2 randomly.
5. Define forward propagation function.
6. Define backward propagation function with weight updates.
7. Train the MLP for 14500 epochs with learning rate 0.001.
8. Print loss every 1000 epochs to track convergence.
9. Test on XOR inputs [0,0], [0,1], [1,0], [1,1] and display outputs.

Code:

```
import pandas as pd
import numpy as np
data = pd.read_csv("XOR_Dataset.csv")

X = data[['X', 'Y']].values
y = data[['Z']].values

print(X)
print(y)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)
np.random.seed(42)

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

W1 = np.random.rand(input_neurons, hidden_neurons)
W2 = np.random.rand(hidden_neurons, output_neurons)

b1 = np.random.rand(1, hidden_neurons)
b2 = np.random.rand(1, output_neurons)

print("W1:", W1)
print("W2:", W2)
print("b1:", b1)
print("b2:", b2)
def forward(X):
    global z1, a1, z2, a2

    z1 = np.dot(X, W1) + b1
    a1 = sigmoid(z1)

    z2 = np.dot(a1, W2) + b2
    a2 = sigmoid(z2)

    return a2
def backward(X, y, output, learning_rate):
    global W1, W2, b1, b2
```

```
    error = y - output
    d_output = error * sigmoid_derivative(output)

    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(a1)

    W2 += a1.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
epochs = 14500
learning_rate = 0.001

for i in range(epochs):
    output = forward(X)
    backward(X, y, output, learning_rate)

    if i % 1000 == 0:
        loss = np.mean(np.square(y - output))
        print(f"Epoch {i}, Loss: {loss}")
print("\n========== XOR_GATE ==========")
print("Final Predictions Approximately:")
print(output)
test = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
print("\n========== XOR_GATE ==========")
print("Test Output Approximately:\n", forward(test))
```

Output:

```
[[0 0]
 [0 1]
 [1 1]
 ...
 [1 1]
 [1 1]
 [1 1]]
[[0]
 [1]
 [0]
 ...
 [0]
 [0]
 [0]]
W1: [[0.37454012 0.95071431]
 [0.73199394 0.59865848]]
W2: [[0.15601864]
 [0.15599452]]
b1: [[0.05808361 0.86617615]]
b2: [[0.60111501]]
Epoch 0, Loss: 0.28619735506826
Epoch 1000, Loss: 0.0007280039634271512
Epoch 2000, Loss: 0.00030333705009284213
Epoch 3000, Loss: 0.00018964029379510061
Epoch 4000, Loss: 0.00013742532286547228
Epoch 5000, Loss: 0.00010755468584735715
Epoch 6000, Loss: 8.825354205089723e-05
Epoch 7000, Loss: 7.477219685109076e-05
Epoch 8000, Loss: 6.483153877344441e-05
Epoch 9000, Loss: 5.720309069523176e-05
Epoch 10000, Loss: 5.116678783278685e-05
Epoch 11000, Loss: 4.627290849808972e-05
Epoch 12000, Loss: 4.222622701423952e-05
Epoch 13000, Loss: 3.882499109583778e-05
Epoch 14000, Loss: 3.592668292272415e-05
========== XOR_GATE ==========
Final Predictions Approximately:
[[0.00670956]
```

```
 [0.99443146]
 [0.00566962]
 ...
 [0.00566962]
 [0.00566962]
 [0.00566962]]
========== XOR_GATE ==========
Test Output Approximately:
 [[0.00670932]
 [0.99443166]
 [0.99445307]
 [0.00566942]]
```

Result:

The MLP successfully learned the XOR function using backpropagation. After 14500 epochs, loss reduced to ~3.5×10^{-5}. Predictions: [0,0]→0.007, [0,1]→0.994, [1,0]→0.994, [1,1]→0.006, confirming correct XOR behaviour.

Ex No: 6 Date: 30/03/2026	Face Recognition using SVM

Aim:

To build a face recognition system using Support Vector Machine (SVM) with Haar cascade face detection and evaluate classification performance.

Algorithm:

1. Import cv2, numpy, sklearn SVC, LabelEncoder, metrics.
2. Load training images from dataset/train/ folders (one folder per person).
3. Convert images to grayscale; detect faces using Haar cascade.
4. Resize each detected face to 100×100 and flatten to 1D vector.
5. Encode person labels using LabelEncoder.
6. Split data into train/test sets (80:20, stratified).
7. Train SVM classifier with linear kernel.
8. For each test image, detect face and predict using trained SVM.
9. Compute accuracy, confusion matrix, and classification report.

Code:

```
import os
import cv2
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
DATASET_PATH = "data/train/"

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
                                     'haarcascade_frontalface_default.xml')

X = []
y = []

for person in os.listdir(DATASET_PATH):
    person_path = os.path.join(DATASET_PATH, person)

    if not os.path.isdir(person_path):
        continue

    for img_name in os.listdir(person_path):
        img_path = os.path.join(person_path, img_name)

        img = cv2.imread(img_path)
        if img is None:
            continue

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        for (x, y_, w, h) in faces:
            face = gray[y_:y_+h, x:x+w]
            face = cv2.resize(face, (100, 100))

            X.append(face.flatten())
            y.append(person)

print(f"Dataset Loaded: {len(X)} samples")
X = np.array(X)
y = np.array(y)
```

```
le = LabelEncoder()
y_encoded = le.fit_transform(y)

print("Classes:", le.classes_)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
print("X shape:", X.shape)
print("y shape:", y.shape)
print("Unique labels:", np.unique(y))
model = SVC(kernel='linear', probability=True)
model.fit(X_train, y_train)

print("Training Completed")
val_path = "data/test/"

y_true = []
y_pred = []

total_images = 0
detected_faces = 0

for person in os.listdir(val_path):
    person_path = os.path.join(val_path, person)

    if not os.path.isdir(person_path):
        continue

    for img_name in os.listdir(person_path):
        img_path = os.path.join(person_path, img_name)
        total_images += 1

        img = cv2.imread(img_path)
        if img is None:
            print("Failed to load:", img_path)
            continue

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        # Improved detection
        faces = face_cascade.detectMultiScale(gray, 1.1, 3)

        # Fallback (VERY IMPORTANT)
        if len(faces) == 0:
            print("No face detected:", img_path)
            h, w = gray.shape
            faces = [(0, 0, w, h)]  # fallback full image

        for (x, y_, w, h) in faces:
            detected_faces += 1

            face = gray[y_:y_+h, x:x+w]
            face = cv2.resize(face, (100, 100))
            face = face.flatten().reshape(1, -1)

            pred = model.predict(face)
            label = le.inverse_transform(pred)[0]

            y_true.append(person)
            y_pred.append(label)

# Debug Summary
print("\n===== DEBUG SUMMARY =====")
print("Total images:", total_images)
print("Faces processed:", detected_faces)
print("Predictions made:", len(y_pred))

# If still empty → stop early
```

```
if len(y_pred) == 0:
    raise ValueError("No predictions made. Check face detection or dataset.")

print("\nAccuracy:", accuracy_score(y_true, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_true, y_pred))
print("\nClassification Report:\n", classification_report(y_true, y_pred))
```

Output:

```
Dataset Loaded: 75 samples
Classes: ['ben_afflek' 'elton_john' 'jerry_seinfeld' 'madonna' 'mindy_kaling']
X shape: (75, 10000)
y shape: (75,)
Unique labels: ['ben_afflek' 'elton_john' 'jerry_seinfeld' 'madonna'
'mindy_kaling']
Training Completed
===== DEBUG SUMMARY =====
Total images: 25
Faces processed: 28
Predictions made: 28

Accuracy: 0.6785714285714286

Confusion Matrix:
 [[4 0 0 0 1]
 [1 2 2 0 0]
 [0 0 6 0 0]
 [1 0 2 2 0]
 [2 0 0 0 5]]

Classification Report:
                 precision    recall  f1-score   support

    ben_afflek       0.50      0.80      0.62         5
    elton_john       1.00      0.40      0.57         5
jerry_seinfeld       0.60      1.00      0.75         6
       madonna       1.00      0.40      0.57         5
  mindy_kaling       0.83      0.71      0.77         7

      accuracy                           0.68        28
     macro avg       0.79      0.66      0.66        28
  weighted avg       0.78      0.68      0.67        28
```

Result:

Face recognition using SVM achieved 67.86% accuracy on 5 celebrity classes. The model was trained on 75 images and tested on 28 face samples. jerry_seinfeld had the highest recall (100%), while elton_john and madonna had lower recall (40%).

Ex No: 7 Date: 31/03/2026	Decision Tree Classifier

Aim:

To implement a Decision Tree Classifier with preprocessing and feature encoding to predict student pass/fail results from the Student Performance dataset.

Algorithm:

1. Import pandas, matplotlib, sklearn DecisionTreeClassifier.
2. Load student-por.csv dataset.
3. Create binary target: result = 1 if G3 >= 10, else 0.
4. Apply one-hot encoding using pd.get_dummies().
5. Drop intermediate grade columns G1 and G2 from features.
6. Split data into 80% train, 20% test.
7. Train DecisionTreeClassifier with max_depth=5 and balanced class weights.
8. Predict on test set.
9. Evaluate using accuracy, classification report.
10. Visualize the decision tree using plot_tree().

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
df = pd.read_csv("student-por.csv")

df.head()
df['result'] = df['G3'].apply(lambda x: 1 if x >= 10 else 0)
df = pd.get_dummies(df, drop_first=True)
X = df.drop(['G3', 'result'], axis=1)
X = X.drop(['G1', 'G2'], axis=1)
y = df['result']
print(X.dtypes)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(max_depth=5, class_weight='balanced',
random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4)*100, "%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))
plt.figure(figsize=(15,10))
plot_tree(model, feature_names=X.columns, class_names=['Fail','Pass'], filled=True)
plt.show()
```

Output:

```
school sex  age address famsize Pstatus  Medu  Fedu     Mjob      Fjob  ...  \
0     GP   F   18       U     GT3       A     4     4  at_home   teacher  ...
1     GP   F   17       U     GT3       T     1     1  at_home     other  ...
2     GP   F   15       U     LE3       T     1     1  at_home     other  ...
3     GP   F   15       U     GT3       T     4     2   health  services  ...
4     GP   F   16       U     GT3       T     3     3    other     other  ...

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4        3      4     1     1      3        4   0  11  11
1      5        3      3     1     1      3        2   9  11  11
2      4        3      2     2     3      3        6  12  13  12
```

```
3      3        2      2     1     1      5        0  14  14  14
4      4        3      2     1     2      5        0  11  13  13

[5 rows x 33 columns]
age                  int64
Medu                 int64
Fedu                 int64
traveltime           int64
studytime            int64
failures             int64
famrel               int64
freetime             int64
goout                int64
Dalc                 int64
Walc                 int64
health               int64
absences             int64
school_MS             bool
sex_M                 bool
address_U             bool
famsize_LE3           bool
Pstatus_T             bool
Mjob_health           bool
Mjob_other            bool
Mjob_services         bool
Mjob_teacher          bool
Fjob_health           bool
Fjob_other            bool
Fjob_services         bool
Fjob_teacher          bool
reason_home           bool
reason_other          bool
reason_reputation     bool
guardian_mother       bool
guardian_other        bool
schoolsup_yes         bool
famsup_yes            bool
paid_yes              bool
activities_yes        bool
nursery_yes           bool
higher_yes            bool
internet_yes          bool
romantic_yes          bool
dtype: object
DecisionTreeClassifier(class_weight='balanced', max_depth=5, random_state=42)
Accuracy: 82.31 %

Classification Report:
              precision    recall  f1-score   support

           0       0.35      0.60      0.44        15
           1       0.94      0.85      0.89       115

    accuracy                           0.82       130
   macro avg       0.64      0.73      0.67       130
weighted avg       0.87      0.82      0.84       130
<Figure size 1500x1000 with 1 Axes>
```

Result:

Decision Tree Classifier achieved 82.31% accuracy on the student performance dataset. The tree with max_depth=5 and balanced class weights effectively classified pass/fail outcomes, with 94% precision for passing students.

Ex No: 8 Date: 07/04/2026	Boosting (Ensemble Learning – AdaBoost)

Aim:

To implement AdaBoost ensemble learning with Decision Tree as base estimator for breast cancer classification.

Algorithm:

1. Import pandas, sklearn AdaBoostClassifier, DecisionTreeClassifier.
2. Load breast-cancer.csv; encode diagnosis: M=1, B=0.
3. Drop unnecessary columns (id).
4. Split features (X) and target (y).
5. Split into 80% train, 20% test.
6. Initialize base model: DecisionTreeClassifier(max_depth=1).
7. Train AdaBoostClassifier with 100 estimators, learning_rate=0.1.
8. Apply probability threshold of 0.4 for final predictions.
9. Evaluate with accuracy, cross-validation (5-fold), confusion matrix, classification report.
10. Test on individual sample predictions.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
df = pd.read_csv("breast-cancer.csv")

df['diagnosis'] = df['diagnosis'].map({'B': 0, 'M': 1})

# View first 5 rows
df.head()
# Drop unnecessary column if exists
if 'id' in df.columns:
    df = df.drop('id', axis=1)

# Convert categorical target to numeric
# Example: M = 1, B = 0
if df['diagnosis'].dtype == 'object':
    df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

df.head()
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
base_model = DecisionTreeClassifier(max_depth=1)

model = AdaBoostClassifier(
    estimator=base_model,
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)
model.fit(X_train, y_train)
y_prob = model.predict_proba(X_test)[:,1]

# Threshold tuning (IMPORTANT)
y_pred = (y_prob > 0.4).astype(int)
train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)
```

```
print(f"Train Accuracy:{train_acc}")
print(f"Test Accuracy :{test_acc}")
scores = cross_val_score(model, X, y, cv=5)

print("Cross-validation scores:", scores)
print("Mean CV accuracy:", scores.mean())
print("Accuracy:\n", round(accuracy_score(y_test, y_pred), 2))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
benign_idx = y_test[y_test == 0].sample(5).index
malignant_idx = y_test[y_test == 1].sample(5).index

indices = list(benign_idx) + list(malignant_idx)

for i in indices:
    sample = X_test.loc[[i]]
    pred = model.predict(sample)[0]
    actual = y_test.loc[i]

    print(f"Sample {i}: Predicted =",
          "Malignant(Cancer Detected)" if pred==1 else "Benign(No Cancer)",
          "| Actual =",
          "Malignant(Cancer Detected)" if actual==1 else "Benign(No Cancer)")
```

Output:

```
id  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302          1        17.99         10.38          122.80     1001.0
1    842517          1        20.57         17.77          132.90     1326.0
2  84300903          1        19.69         21.25          130.00     1203.0
3  84348301          1        11.42         20.38           77.58      386.1
4  84358402          1        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  radius_worst  texture_worst  perimeter_worst  area_worst  \
0  ...         25.38          17.33           184.60      2019.0
1  ...         24.99          23.41           158.80      1956.0
2  ...         23.57          25.53           152.50      1709.0
3  ...         14.91          26.50            98.87       567.7
4  ...         22.54          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave points_worst  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   symmetry_worst  fractal_dimension_worst
0          0.4601                  0.11890
1          0.2750                  0.08902
2          0.3613                  0.08758
3          0.6638                  0.17300
4          0.2364                  0.07678

[5 rows x 32 columns]
diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0          1        17.99         10.38          122.80     1001.0
1          1        20.57         17.77          132.90     1326.0
2          1        19.69         21.25          130.00     1203.0
3          1        11.42         20.38           77.58      386.1
4          1        20.29         14.34          135.10     1297.0
```

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

Result:

AdaBoost classifier achieved 96% accuracy on breast cancer classification. Train accuracy was 98.24%, and mean 5-fold cross-validation accuracy was 96.14%, demonstrating robust generalization. Confusion matrix: [[67,4],[1,42]].

Ex No: 9a Date: 07/04/2026	K-Nearest Neighbors (KNN) Classification

Aim:

To implement K-Nearest Neighbors algorithm to classify penguin sex based on physical measurements, and find the optimal K value.

Algorithm:

1. Import pandas, matplotlib, seaborn, sklearn KNN modules.
2. Load penguins.csv; encode sex column using LabelEncoder.
3. Split features and target; perform 80:20 train-test split (stratified).
4. Scale features using StandardScaler.
5. Find best K by plotting train/test accuracy for K=1 to 15.
6. Train final KNN model with K=5.
7. Predict on train and test sets.
8. Compute accuracy, confusion matrix, classification report.
9. Visualize predicted classes on scatter plot.
10. Predict on custom input; visualize 5 nearest neighbors.

Code:

```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier, NearestNeighbors
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Load Dataset
df = pd.read_csv("penguins.csv")
print("Shape :", df.shape)

# Encode Target Column
le = LabelEncoder()
df["sex"] = le.fit_transform(df["sex"])   # FEMALE / MALE

df.head()
# Split Features and Target
X = df.drop("sex", axis=1)
y = df["sex"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train Shape :", X_train.shape)
print("Test Shape  :", X_test.shape)
# Feature Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
# Find Best K
train_scores = []
test_scores = []

for k in range(1,16):
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    knn.fit(X_train_scaled, y_train)

    train_scores.append(knn.score(X_train_scaled, y_train))
    test_scores.append(knn.score(X_test_scaled, y_test))

plt.figure(figsize=(10,5))
plt.plot(range(1,16), train_scores, marker='o', label="Train Accuracy")
plt.plot(range(1,16), test_scores, marker='o', label="Test Accuracy")
plt.title("Choose Best K")
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
# Train Final KNN Model
# Choose best K (example: 5)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
# Prediction
y_train_pred = knn.predict(X_train_scaled)
y_test_pred  = knn.predict(X_test_scaled)
# Accuracy
train_acc = accuracy_score(y_train, y_train_pred)
test_acc  = accuracy_score(y_test, y_test_pred)

print("Train Accuracy :", round(train_acc*100,2), "%")
print("Test Accuracy  :", round(test_acc*100,2), "%")
# Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
print("Classification Report :\n", classification_report(y_test, y_test_pred,
target_names=le.classes_))
# Visualize Test Data Prediction
plt.figure(figsize=(8,6))

sns.scatterplot(
    x=X_test["body_mass_g"],
    y=X_test["flipper_length_mm"],
    hue=y_test_pred,
    palette="Set1",
    s=100
)

plt.title("KNN Predicted Classes")
plt.xlabel("Body Mass (g)")
plt.ylabel("Flipper Length (mm)")
plt.show()
# Custom Data Prediction
custom_data = pd.DataFrame([{
    "culmen_length_mm":45.2,
    "culmen_depth_mm":17.1,
    "flipper_length_mm":200,
    "body_mass_g":5000
}])
```

```
custom_scaled = scaler.transform(custom_data)

pred = knn.predict(custom_scaled)[0]

print("Predicted Sex :", le.inverse_transform([pred])[0])
# Find 5 nearest neighbors
nn = NearestNeighbors(n_neighbors=5)
nn.fit(X_train_scaled)
distances, indices = nn.kneighbors(custom_scaled)

# Convert scaled data back
train_original = scaler.inverse_transform(X_train_scaled)
custom_original = scaler.inverse_transform(custom_scaled)

# Feature names
features = [
    "culmen_length_mm",
    "culmen_depth_mm",
    "flipper_length_mm",
    "body_mass_g"
]

pairs = list(combinations(range(4), 2))

# Create 6 subplots
fig, axes = plt.subplots(3, 2, figsize=(10, 15))
axes = axes.flatten()

male_mask = (y_train.values == le.transform(["MALE"])[0])
female_mask = (y_train.values == le.transform(["FEMALE"])[0])

for plot_id, (i, j) in enumerate(pairs):

    ax = axes[plot_id]

    # Female points
    ax.scatter(
        train_original[female_mask, i],
        train_original[female_mask, j],
        c='blue',
        alpha=0.45,
        s=25,
        label="Female" if plot_id == 0 else ""
    )

    # Male points
    ax.scatter(
        train_original[male_mask, i],
        train_original[male_mask, j],
        c='red',
        alpha=0.45,
        s=25,
        label="Male" if plot_id == 0 else ""
    )

    # Nearest neighbors
    ax.scatter(
        train_original[indices[0], i],
        train_original[indices[0], j],
        c='lime',
        s=50,
        edgecolors='black',
        label="5 Neighbors" if plot_id == 0 else ""
    )

    # Custom point
    ax.scatter(
        custom_original[:, i],
```

```
        custom_original[:, j],
        c='black',
        marker='*',
        s=50,
        label="Custom Point" if plot_id == 0 else ""
    )

    # Lines to neighbors
    for idx in indices[0]:
        ax.plot(
            [custom_original[0, i], train_original[idx, i]],
            [custom_original[0, j], train_original[idx, j]],
            '--',
            color='gray',
            linewidth=1
        )

    ax.set_title(f"{features[i]} vs {features[j]}")
    ax.set_xlabel(features[i])
    ax.set_ylabel(features[j])

handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc="upper center", ncol=4, fontsize=11)

plt.suptitle("KNN Pairwise Neighbor Visualization", fontsize=16, weight="bold")
plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.show()
```

Output:

```
Shape : (332, 5)
culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g  sex
0              39.1             18.7                181         3750    1
1              39.5             17.4                186         3800    0
2              40.3             18.0                195         3250    0
3              36.7             19.3                193         3450    0
4              39.3             20.6                190         3650    1
Train Shape : (265, 4)
Test Shape  : (67, 4)
<Figure size 1000x500 with 1 Axes>
KNeighborsClassifier()
Train Accuracy : 93.58 %
Test Accuracy  : 95.52 %
<Figure size 600x500 with 2 Axes>
Classification Report :
               precision    recall  f1-score   support

      FEMALE       0.97      0.94      0.95        33
        MALE       0.94      0.97      0.96        34

    accuracy                           0.96        67
   macro avg       0.96      0.95      0.96        67
weighted avg       0.96      0.96      0.96        67
```

Result:

KNN classifier with K=5 achieved 95.52% test accuracy and 93.58% train accuracy on the penguins dataset for gender classification. Custom input [45.2, 17.1, 200, 5000] was predicted as MALE.

Ex No: 9b Date: 15/04/2026	K-Means Clustering

Aim:

To implement K-Means clustering algorithm on the Penguins dataset to discover natural groupings using the Elbow method and silhouette score.

Algorithm:

1. Import pandas, matplotlib, seaborn, sklearn KMeans, silhouette_score.
2. Load penguins.csv; remove label column (sex).
3. Scale features using StandardScaler.
4. Apply Elbow Method: compute WCSS for K=1 to 10.
5. Select optimal K=2 from elbow plot.
6. Train KMeans model with K=2, n_init=10.
7. Assign cluster labels to dataset.
8. Compute Silhouette Score.
9. Visualize clusters with centroids on scatter plot.
10. Predict cluster for custom input data.

Code:

```
# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# Load Dataset
df = pd.read_csv("penguins.csv")

X = df.drop("sex", axis=1)   # remove label column

print("Shape :", X.shape)
X.head()
# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Elbow Method
wcss = []

for i in range(1,11):
    km = KMeans(n_clusters=i, random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss.append(km.inertia_)

plt.figure(figsize=(8,5))
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("No. of Clusters")
plt.ylabel("WCSS")
plt.show()
# Train Final Model
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df["Cluster"] = clusters

df.head()
# Silhouette Score
score = silhouette_score(X_scaled, clusters)
print("Silhouette Score :", round(score,3))
# Cluster Count
print(df["Cluster"].value_counts())
```

```
# Visualization
plt.figure(figsize=(8,6))

# Scatter points
sns.scatterplot(
    x=df["body_mass_g"],
    y=df["flipper_length_mm"],
    hue=df["Cluster"],
    palette="Set1",
    s=100
)

# Get centers from scaled data -> inverse transform
centers = scaler.inverse_transform(kmeans.cluster_centers_)
print(centers)

# body_mass_g index = 3
# flipper_length_mm index = 2

plt.scatter(
    centers[:,3],          # body_mass_g
    centers[:,2],          # flipper_length_mm
    c="black",
    s=200,
    marker="*",
    label="Centroids"
)

plt.title("KMeans Clusters with Cluster Centers")
plt.xlabel("Body Mass (g)")
plt.ylabel("Flipper Length (mm)")
plt.legend()
plt.show()
# Custom Data Prediction
custom_data = pd.DataFrame([{
    "culmen_length_mm":45.2,
    "culmen_depth_mm":17.1,
    "flipper_length_mm":210,
    "body_mass_g":5000
}])

custom_scaled = scaler.transform(custom_data)
pred_cluster = kmeans.predict(custom_scaled)

print("Predicted Cluster :", pred_cluster[0])
```

Output:

```
Shape : (332, 4)
culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g
0              39.1             18.7                181         3750
1              39.5             17.4                186         3800
2              40.3             18.0                195         3250
3              36.7             19.3                193         3450
4              39.3             20.6                190         3650
<Figure size 800x500 with 1 Axes>
culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g     sex  \
0              39.1             18.7                181         3750    MALE
1              39.5             17.4                186         3800  FEMALE
2              40.3             18.0                195         3250  FEMALE
3              36.7             19.3                193         3450  FEMALE
4              39.3             20.6                190         3650    MALE

   Cluster
0        0
1        0
2        0
3        0
```

```
4        0
Silhouette Score : 0.531
Cluster
0    213
1    119
Name: count, dtype: int64
[[  42.03943662   18.35774648  191.89201878 3711.50234742]
 [  47.56806723   14.99663866  217.23529412 5092.43697479]]
<Figure size 800x600 with 1 Axes>
```

Result:

K-Means clustering with K=2 achieved a Silhouette Score of 0.531 on the penguins dataset. Cluster 0 had 213 samples, Cluster 1 had 119 samples. Custom input [45.2, 17.1, 210, 5000] was assigned to Cluster 1.

Ex No: 10 Date: 15/04/2026	Principal Component Analysis (PCA)

Aim:

To apply PCA for dimensionality reduction on the Penguins dataset from 4D to 2D and train a KNN classifier on the reduced feature space.

Algorithm:

1. Import pandas, sklearn PCA, StandardScaler, KNN, LabelEncoder.
2. Load penguins.csv; drop null values.
3. Encode target column (sex) using LabelEncoder.
4. Scale features using StandardScaler.
5. Apply PCA with n_components=2 to reduce 4D to 2D.
6. Print explained variance ratio and total variance covered.
7. Visualize PCA components using scatter plot coloured by sex.
8. Print PCA component loadings.
9. Train KNN (K=5) on PCA-transformed data.
10. Scale and transform custom input; predict class using KNN; visualize nearest neighbors.

Code:

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier, NearestNeighbors
# Load Dataset
df = pd.read_csv("penguins.csv")
df = df.dropna()

print(df.shape)
df.head()
# Encode target column (sex)
le = LabelEncoder()
df["sex"] = le.fit_transform(df["sex"])

# Features and Target
X = df.drop("sex", axis=1)
y = df["sex"]

X.head()
# Standard Scaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Apply PCA for dimensionality reduction (4D to 2D)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
# Variance ratio
print("Explained Variance Ratio :", pca.explained_variance_ratio_)
print("Total Variance Covered :", round(sum(pca.explained_variance_ratio_) * 100,
2), "%")
# Visualize PCA result
plt.figure(figsize=(9,7))

sns.scatterplot(
    x=X_pca[:,0],
    y=X_pca[:,1],
    hue=y,
    palette={0:"blue", 1:"red"},
    s=100
```

```
)

plt.title("PCA Visualization of Penguins Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(labels=["Female", "Male"])
plt.show()
# Print the loadings
loadings = pd.DataFrame(
    pca.components_.T,
    columns=["PC1", "PC2"],
    index=X.columns
)

loadings
# Train KNN model on PCA data
knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(X_pca, y)
# Create custom input data
custom_data = pd.DataFrame([{
    "culmen_length_mm": 45.2,
    "culmen_depth_mm": 17.1,
    "flipper_length_mm": 210,
    "body_mass_g": 4500
}])

# Scale and transform custom data using PCA
custom_scaled = scaler.transform(custom_data)
custom_pca = pca.transform(custom_scaled)

print("PC1 =", round(custom_pca[0][0], 3))
print("PC2 =", round(custom_pca[0][1], 3))
# Predict class of custom input
pred = knn_pca.predict(custom_pca)[0]

print("Predicted Class :", le.inverse_transform([pred])[0])
# Plot dataset with custom point and 5 nearest neighbors
# Find 5 nearest points in PCA space
nn = NearestNeighbors(n_neighbors=5)
nn.fit(X_pca)

distances, indices = nn.kneighbors(custom_pca)

plt.figure(figsize=(9,7))

# Plot original PCA points
sns.scatterplot(
    x=X_pca[:,0],
    y=X_pca[:,1],
    hue=y,
    palette={0:"blue", 1:"red"},
    s=75
)

# Plot nearest neighbors
plt.scatter(
    X_pca[indices[0],0],
    X_pca[indices[0],1],
    c="lime",
    s=75,
    edgecolors="black",
    label="5 Nearest Points"
)

# Plot custom point
plt.scatter(
    custom_pca[:,0],
    custom_pca[:,1],
```

```
    c="black",
    marker="*",
    s=75,
    label="Custom Input"
)

# Draw lines from custom point to neighbors
for idx in indices[0]:
    plt.plot(
        [custom_pca[0,0], X_pca[idx,0]],
        [custom_pca[0,1], X_pca[idx,1]],
        '--',
        color='gray',
        linewidth=1.5
    )

# Predicted class text
plt.text(
    custom_pca[0][0] + 0.1,
    custom_pca[0][1],
    f"Predicted: {le.inverse_transform([pred])[0]}",
    fontsize=11,
    weight="bold"
)

plt.title("PCA Custom Prediction with 5 Nearest Points")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(labels=["Female", "Male", "5 Nearest Points", "Custom Input"])
plt.show()
```

Output:

```
(332, 5)
culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g     sex
0              39.1             18.7                181         3750    MALE
1              39.5             17.4                186         3800  FEMALE
2              40.3             18.0                195         3250  FEMALE
3              36.7             19.3                193         3450  FEMALE
4              39.3             20.6                190         3650    MALE
culmen_length_mm  culmen_depth_mm  flipper_length_mm  body_mass_g
0              39.1             18.7                181         3750
1              39.5             17.4                186         3800
2              40.3             18.0                195         3250
3              36.7             19.3                193         3450
4              39.3             20.6                190         3650
Explained Variance Ratio : [0.68713344 0.19641564]
Total Variance Covered : 88.35 %
<Figure size 900x700 with 1 Axes>
PC1       PC2
culmen_length_mm   0.453174  0.604990
culmen_depth_mm   -0.398518  0.792959
flipper_length_mm  0.576880  0.003798
body_mass_g        0.550478  0.072032
KNeighborsClassifier()
PC1 = 0.681
PC2 = 0.138
```

Result:

PCA reduced the Penguins dataset from 4D to 2D while retaining 88.35% of total variance (PC1: 68.71%, PC2: 19.64%). KNN trained on PCA features correctly predicted custom input as MALE with PC1=0.681, PC2=0.138.